

graphify

Repository Analysis Report

Build semantic knowledge graphs from code repos to help AI assistants navigate large codebases more effectively.

■ Files	25	■ Lines	13,122
■ Architecture	CLI Tool / Library	■ Provider	anthropic/claude-sonnet-4-20250514
■ Date	April 09, 2026	■ Languages	Python (18), Markdown (7)

Table of Contents

1.	Executive Summary
2.	Technology Stack
3.	Architecture Overview
4.	System Architecture Diagram
5.	Data Flow Diagram
6.	Component Diagram
7.	Key Components
8.	API Endpoints
9.	Code Quality & Security
10.	File-by-File Analysis
11.	Improvement Recommendations
12.	Repository File Tree

1. Executive Summary

Graphify is a Python CLI tool that builds semantic knowledge graphs from code repositories to enhance LLM-assisted code navigation. It parses source code using tree-sitter into AST nodes, assembles a NetworkX graph, detects communities via the Leiden algorithm, and exposes BFS/DFS traversal for AI coding assistants.

Design Patterns

- Strategy Pattern: LanguageConfig dataclass allows plugging in different language parsers

Entry Points

- graphify/__main__.py: CLI entry point with install/query/benchmark commands

2. Technology Stack

Technologies & Frameworks

Python 3.9+

NetworkX

tree-sitter

Language Distribution

Language	Files	Share
Python	18	72.0%
Markdown	7	28.0%

3. Architecture Overview

Architecture Style: **CLI Tool / Library**

Data Flow

1. User invokes CLI
2. Files are scanned and parsed by `extract.py`
3. Nodes/edges assembled in `build.py`
4. Communities detected by `cluster.py`
5. Report generated to `graphify-out/`

4. System Architecture Diagram

High-level system components and their relationships.

[Architecture - diagram code]

```
graph TD
  CLI[CLI __main__.py] --> Build[build.py]
  Build --> Extract[extract.py]
  Build --> Cluster[cluster.py]
  Build --> Analyze[analyze.py]
  Analyze --> Report[report.py]
  CLI --> Serve[serve.py BFS/DFS]
```

5. Data Flow Diagram

Main request / data flow through the system.

[Data Flow - diagram code]

```
sequenceDiagram
    participant User
    participant CLI
    participant Extract
    participant Build
    participant Cluster
    User->>CLI: graphify /path
    CLI->>Extract: parse files
    Extract-->>CLI: nodes+edges
    CLI->>Build: build_graph()
    Build->>Cluster: detect_communities()
    Cluster-->>Build: community_map
    Build-->>User: graphify-out/
```

6. Component Diagram

Module dependencies and internal structure.

[Components - diagram code]

```
graph LR
  subgraph Ingestion
    I[ingest.py]
    E[extract.py]
  end
  subgraph Core
    B[build.py]
    C[cluster.py]
  end
  subgraph Output
    A[analyze.py]
    R[report.py]
    W[wiki.py]
  end
  Ingestion --> Core
  Core --> Output
```


7. Key Components

AST Extractor

Parses code using tree-sitter into nodes and edges

Files: `graphify/extract.py`

Graph Builder

Assembles knowledge graph from extracted nodes

Files: `graphify/build.py`

Cluster Analyzer

Detects communities using Leiden algorithm

Files: `graphify/cluster.py`

9. Code Quality & Security

Code Quality Observations

- Well-structured modules: Clear separation of extraction, building, clustering, and serving

Security Considerations

- Safe URL Fetching: security.py wraps all HTTP calls with validation to prevent SSRF

10. File-by-File Analysis

11. Improvement Recommendations

1. Add async support: Large repos would benefit from async file processing

12. Repository File Tree

```
graphify/
■■■&nbsp;graphify/
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;__init__.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;__main__.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;analyze.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;benchmark.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;build.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;cache.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;cluster.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;detect.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;export.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;extract.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;hooks.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;ingest.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;manifest.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;report.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;security.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;serve.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;skill-claw.md
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;skill-codex.md
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;skill-droid.md
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;skill-opencode.md
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;skill-trae.md
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;skill-windows.md
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;skill.md
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;validate.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;watch.py
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;■■■&nbsp;wiki.py
```

Appendix: Agentic Analysis Trace

This report was generated by an AI agent that autonomously explored the repository, decided which files to read, and recorded findings — rather than following a fixed script.

Provider	anthropic/claude-sonnet-4-20250514
Total Agent Steps	0
Total Tool Calls	18
Analysis Time	47.3s
Files Scanned	25
Findings Recorded	25